



OOSC 4.0 Hackathon



- Problem Statement - 2

- Problem Statement Title -

AI for Equitable Education Access

Build an AI system that closes part of the education access gap, for students, teachers, or both, using open or public educational content as its knowledge base.

- Problem Statement Category -
SOFTWARE

- Team Name - ***GobLins***

- Team Members :

1. Abhishek

2. Harshdeep Singh

3. Gedala Mukesh Kumar



OOSC 4.0 Hackathon



Executive Project Overview

Scholarship Hunt & EduAccess

A full-stack platform that helps underserved students find scholarships and access learning opportunities through a secure, role-based system connecting Students and Scholarship Providers.

Current Phase : Core MVP

- Tech Stack: MERN (MongoDB, Express.js, React.js, Node.js), Redux Toolkit, Tailwind CSS.
- Architecture: Modular client-server architecture using RESTful APIs.
- Security: JWT authentication, HTTP-only cookies, bcrypt password hashing, and OTP email verification.
- Media Storage: Memory-buffered uploads streamed directly to Cloudinary CDN.
- Access Control: Secure role-based access for Students and Scholarship Providers.

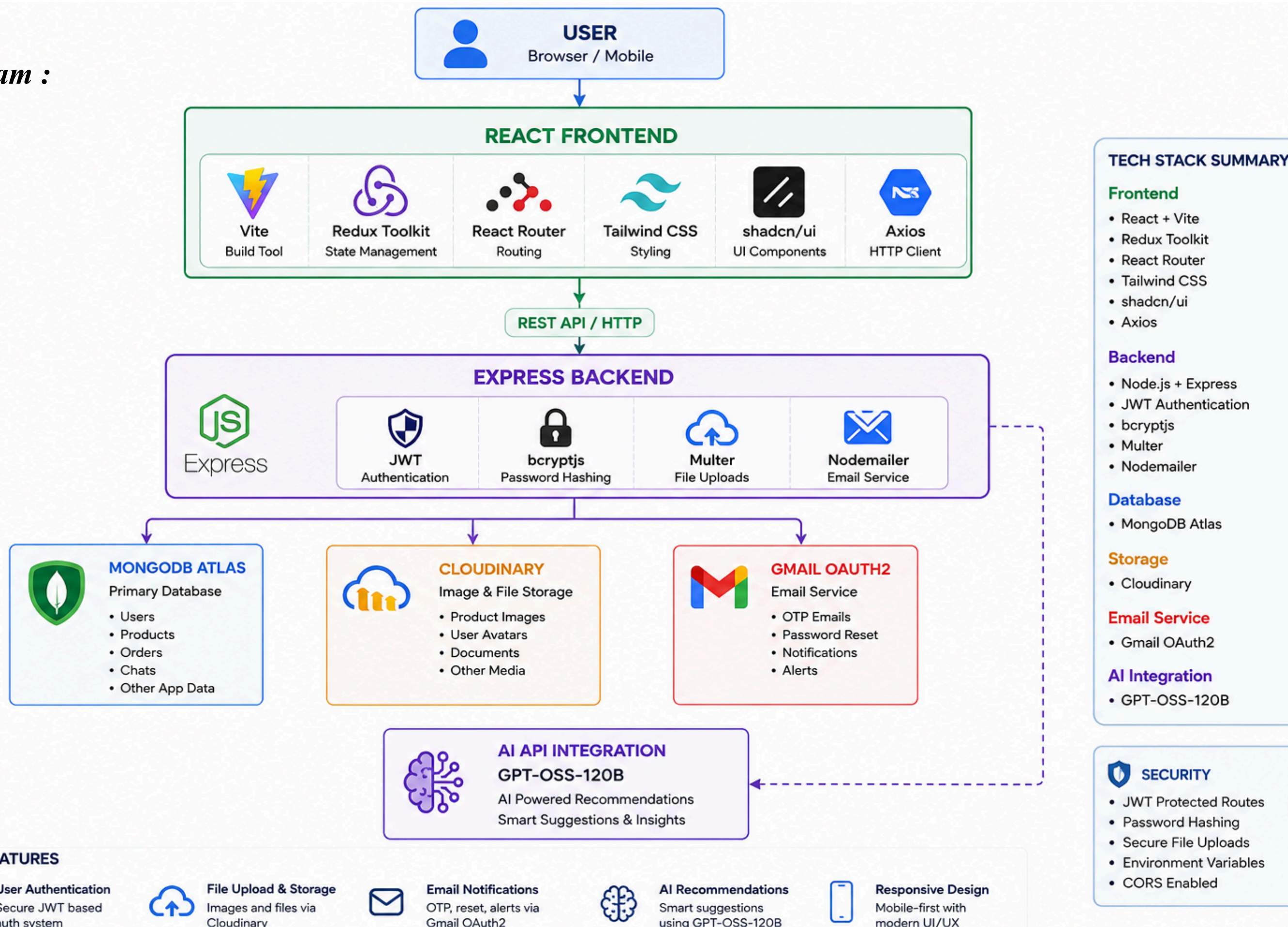
Future AI Roadmap

- AI Doubt-Solving Agent: Step-by-step learning support using trusted textbooks in regional languages.
- AI Eligibility Matcher: Recommends scholarships based on a student's profile and eligibility.
- Adaptive Practice Engine: Generates personalized quizzes to improve weak areas.
- Teacher Insight Dashboard: Predictive analytics to identify students who need academic support.

OOSC 4.0 Hackathon



Architecture diagram :





OOSC 4.0 Hackathon



Technical Architecture & Core System Working :

1. Tech Stack & Architecture

- Architecture: Decoupled client-server model using modular RESTful APIs.
- Frontend: React.js, Redux Toolkit, Tailwind CSS, Axios.
- Backend & Database: Node.js, Express.js, MongoDB Atlas with Mongoose.
- Services: Cloudinary for media storage and Nodemailer for email/OTP services.

2. End-to-End Workflow

- Registration: User submits details → Multer processes files in memory → Files are uploaded directly to Cloudinary → Password is secured using bcryptjs.
- Profile & Roles: Role-based access for Students and Scholarship providers with protected routes and flexible profile updates.

3. Security

- XSS & CSRF Protection: JWT stored in HTTP-only, SameSite: Strict cookies.
- Data Protection: Mongoose schema validation helps maintain secure and consistent data.

4. Scalability & Future AI

Modular structure allows easy integration of RAG-based doubt solving and AI-powered scholarship matching in Phase 2.